

Introduction to Jenkins

Module 2: Jenkins Overview



Module Topics

- Jenkins in modern DevOps ecosystems
- Jenkins LTS architecture and components
- Plugins, security model, and governance
- Controller/agent overview
- SCM-driven automation (GitHub, GitLab)
- Installing Jenkins LTS
- Running Jenkins in Docker
- Configuration-as-Code (JCasC)
- Initial setup wizard and global tools



Origins

- In the early 2000s, most software teams
 - Integrated code infrequently
 - Ran builds manually
 - Discovered integration problems late in the cycle
- In 2004 by Kohsuke Kawaguchi at Sun Microsystems created the Hudson project
 - Open-source written in Java
 - Designed to automate builds and tests
 - Integrated well with Java build tools like Ant and Maven
 - Easy to set up and xxtensible
 - Focused on automation rather than process enforcement



Hudson to Jenkins (2011)

- In 2010–2011, Oracle acquired Sun Microsystems
 - This led to disagreements between the Hudson community and Oracle regarding:
 - *Project governance*
 - *Trademark ownership*
 - *Contribution control*
- As a result
 - The community forked Hudson
 - The new project was named Jenkins
 - Most contributors and users moved to Jenkins
- From this point on
 - Jenkins became the community-driven project
 - Hudson gradually declined in relevance



Job-Centric CI (2011–2014)

- Early versions of Jenkins focused on:
 - Freestyle jobs
 - UI-driven configuration
 - Scripted build steps
- Characteristics of early Jenkins
 - Configuration stored on the server
 - Pipelines defined through the web UI
 - Heavy reliance on plugins
 - Less reproducible builds
- This reflected the state of CI at the time
 - Automation existed, but IaC was not common



Pipeline as Code (2014–2016)

- As DevOps practices matured, teams demanded
 - Version-controlled build definitions
 - Reproducibility
 - Better support for complex workflows
- This led to the introduction of the Jenkins Pipeline and Jenkinsfile
- Innovations
 - Pipelines defined in code
 - Stored in Git repositories
 - Reviewed like application code
 - Support for stages, parallelism, and conditions
- This marked a major evolution in Jenkins' design philosophy



DevOps Era (2016–Present)

- Jenkins evolved to support
 - Microservices
 - Containers and Docker
 - Kubernetes-based build agents
 - Cloud-native workflows
- Notable Trends
 - Declarative pipelines simplified syntax
 - Dynamic agents replaced static build servers
 - Integration with modern SCM platforms (GitHub, GitLab)
- Jenkins shifted from being “a CI tool” to a general-purpose automation engine



Jenkins Today

- Why Jenkins is still relevant
 - Extreme flexibility
 - Large plugin ecosystem
 - Strong support for legacy and complex systems
- Challenges
 - Requires operational effort
 - Plugin compatibility management
 - Competition from managed CI/CD platforms
- Despite newer tools, Jenkins remains widely used in:
 - Enterprises
 - Regulated environments
 - Complex, heterogeneous systems



GitLab and GitHub

- GitHub Actions and GitLab CI/CD are modern alternatives to Jenkin' pipelines
 - These are integrated pipeline platforms built directly into their Git hosting services
- These repository tools treat CI/CD as a native feature of the repository
 - Jenkins = external automation engine
 - GitHub/GitLab = pipelines embedded in the SCM platform
- The same basic pipeline concepts apply to these the same as Jenkins
 - The syntax and format of the configuration files
- These turn GitHub and GitLab into orchestration engines



GitHub Actions

- GitHub Actions is GitHub's built-in automation and CI/CD platform
 - Pipelines are defined using YAML workflow files stored in the repository.
- Key characteristics
 - Tight integration with GitHub repositories
 - Event-driven (push, pull request, tag, issue events)
 - Pipelines live in `.github/workflows`
 - Uses hosted runners or self-hosted runners
- Conceptual flow
 - GitHub event → Action workflow → Jobs → Steps
- Strengths
 - Zero infrastructure to manage
 - Simple setup for GitHub users
 - Strong marketplace of reusable actions



GitLab CI/CD

- GitLab CI/CD is a first-class feature of GitLab, not an add-on.
 - Pipelines are defined in a single file: “.gitlab-ci.yml”
- Key characteristics
 - CI/CD tightly integrated with GitLab repos
 - Built-in artifact storage and environment tracking
 - Native support for Docker and Kubernetes
 - GitLab Runners execute jobs
- Conceptual flow
 - Git commit → GitLab pipeline → Stages → Jobs
- Strengths
 - Very cohesive developer experience
 - Strong DevOps lifecycle integration
 - Minimal configuration overhead



Comparison

- Configuration style contrast
- Jenkins
 - Highly flexible
 - Scriptable pipelines (Groovy)
 - Powerful but verbose
 - Requires plugin management
- GitHub / GitLab
 - Opinionated YAML syntax
 - Simpler mental model
 - Less customization, faster onboarding
 - Platform constraints apply



Comparison

- Jenkins is often chosen when
 - You need extreme customization
 - You support many SCM systems
 - You have complex legacy workflows
 - You control infrastructure
- GitHub / GitLab pipelines are often chosen when
 - Repositories are already hosted on the platform
 - Teams want fast setup
 - Minimal operational overhead is desired
 - Standard CI/CD patterns are sufficient

Which continuous integration (CI) tool do you regularly use in your company?

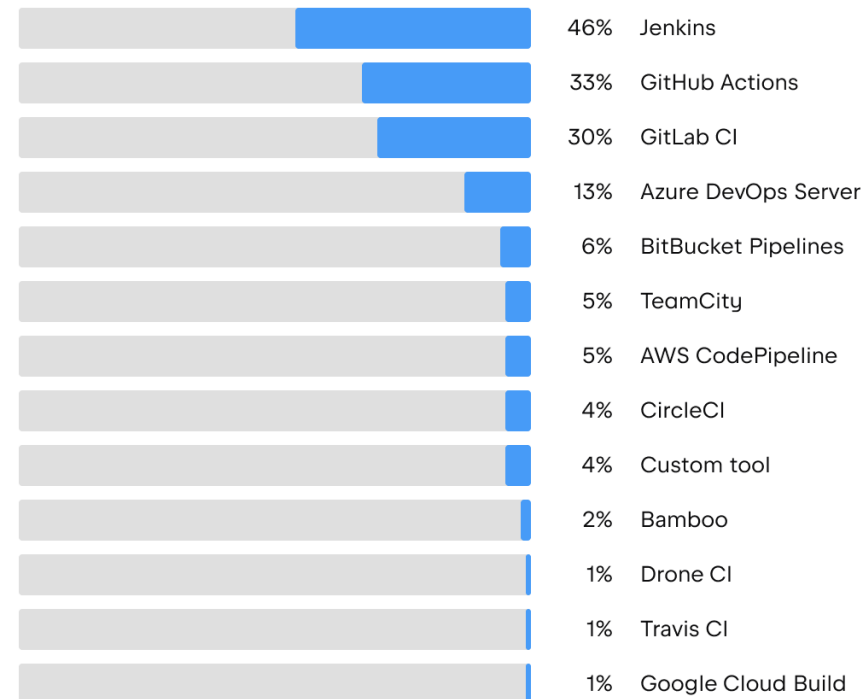


Image Credit: <https://blog.jetbrains.com/teamcity/2023/08/how-to-choose-cicd-tool/>

Modern DevOps Ecosystems

- Jenkins is an automation engine that
 - Listens to SCM events
 - Executes pipelines
 - Orchestrates execution of builds, tests, scans, and deployments
 - Integrates with many external tools
 - *SCM: GitHub, GitLab, Bitbucket*
 - *Build tools: Maven, Gradle, npm, pip*
 - *Containers: Docker, Podman*
 - *Kubernetes: Jenkins Kubernetes plugin*
 - *Quality & Security: SonarQube, Snyk, Trivy*
 - *Secrets: Vault, cloud KMS systems*
- Jenkins is usually the best choice when
 - The pipeline needs to integrate many tools
 - Full control over pipeline workflows is required
 - Operational needs span hybrid or multi-cloud environments



Jenkins Release Model

- Jenkins has two release lines
- Weekly releases
 - New features
 - Faster change
 - Higher risk
- LTS (Long-Term Support)
 - Stability-first
 - Security patches backported
 - Recommended for production

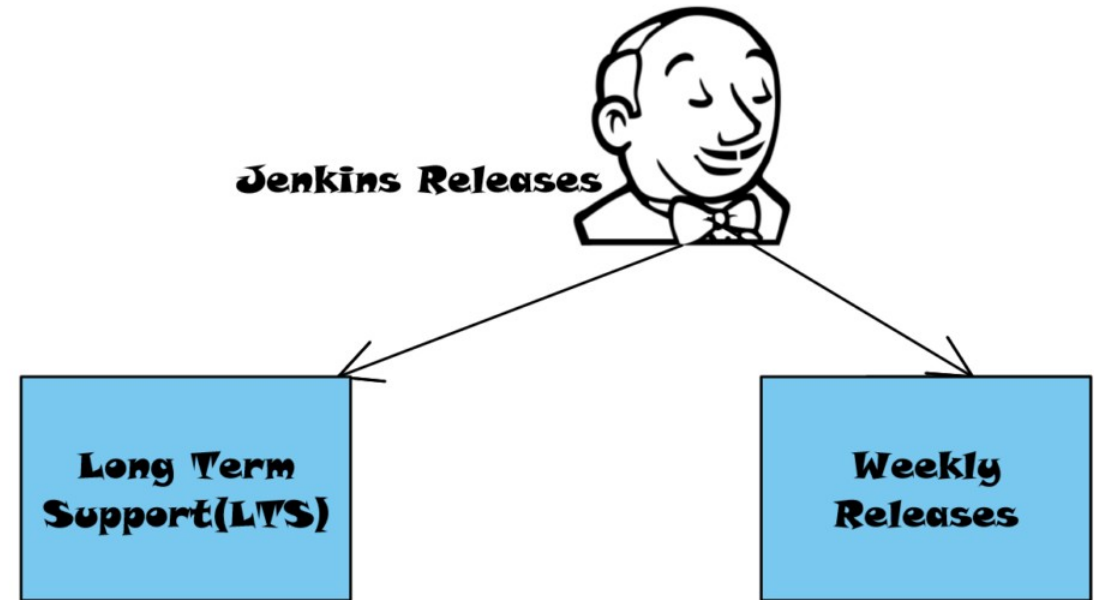


Image Credit: <https://www.testingdocs.com/category/jenkins/>

Jenkins Weekly Releases

- Released every week and include
 - New features
 - Bug fixes
 - Internal changes
- Represent the latest state of Jenkins development
- Typically used by
 - Plugin developers
 - Jenkins core contributors
 - Early adopters testing new features
 - Teams validating future LTS upgrades
- Not meant for
 - Production controllers
 - Regulated environments



Jenkins LTS

- Recommended for production environments
- Plugins are tested primarily against LTS
- Predictable upgrade cycles
 - Allows for enterprise planning for moving to newer versions
- Security patches are back ported from the weekly build
 - Ensures that security issues are addressed in LTS
- Design assumes long running controller



Long Running Controller

- The controller
 - Maintains persistent state on disk
 - Accumulates configuration over time
- Retains:
 - Job definitions
 - Build history
 - Security credentials
 - Plugin data
 - User configuration
- Different design architecture than
 - Stateless web services
 - Ephemeral CI runners
 - Serverless architectures



Long Running Controller

- Jenkins controllers
 - Retain long build histories
 - Preserve logs for auditing
 - Track trends over time
 - Examples
 - *Test flakiness trends*
 - *Build duration changes*
 - *Deployment frequency*
- This historical continuity only works if
 - The controller persists
 - The file system remains stable



Jenkins Architecture

- Jenkins controller
 - Central brain of Jenkins
 - Responsibilities
 - *Web UI*
 - *REST API*
 - *Job and pipeline definitions*
 - *Build scheduling and queue management*
 - *Plugin management*
 - *Credential storage*
 - *User authentication and authorization*
 - *Should not run heavy builds*
 - Referred to as the master node in older versions of Jenkins

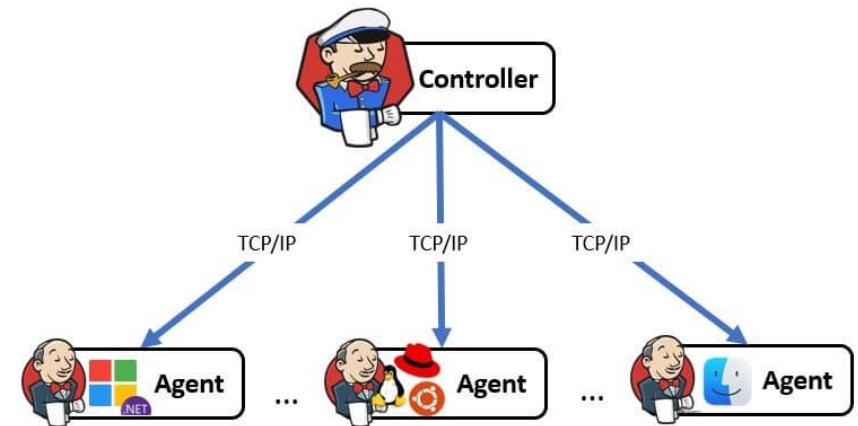


Image Credit: <https://naiwaen.debuggingsoft.com/2022/06/jenkins-controller-and-agents-architecture/>

Jenkins Architecture

- Jenkins controller should NOT
 - Run heavy builds
 - Compile large projects
 - Execute tests
- Jenkins controller is idle most of the time

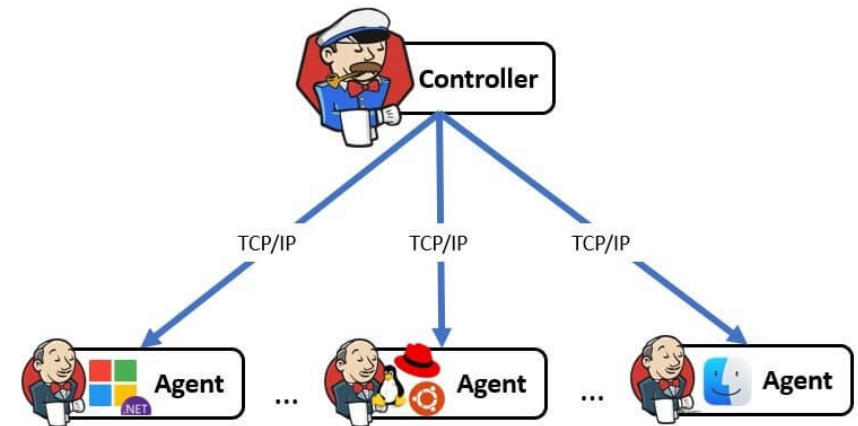


Image Credit: <https://naiwaen.debuggingsoft.com/2022/06/jenkins-controller-and-agents-architecture/>

Internal Controller Components

- Web application
 - Runs as a Java web app
 - Embedded Jetty servlet container
 - UI + API share the same backend
- Job configuration store
 - Jobs stored as XML on disk
 - Pipelines stored either:
 - *Inline (legacy)*
 - *From SCM (preferred)*
- Build queue
 - Pending builds waiting for executors
 - Queue logic decides
 - *When a job runs*
 - *On which agent*
 - *With which constraints*
- Executor management
 - Executors represent capacity, think of them as CPUs
 - Executors live on agents

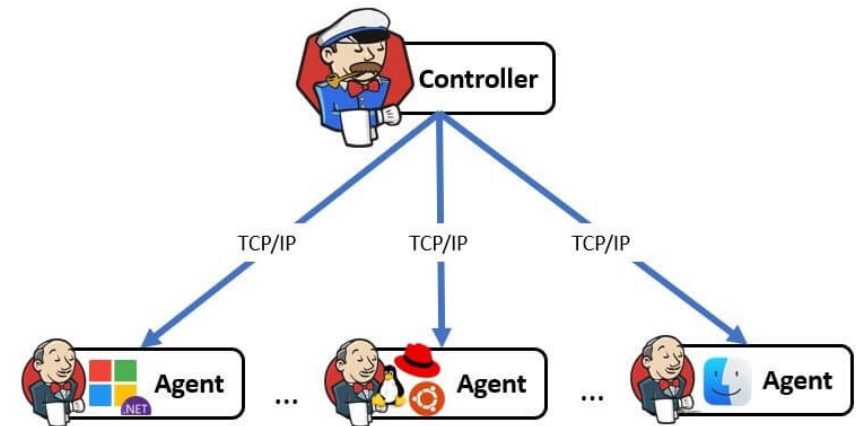


Image Credit: <https://naiwaen.debuggingsoft.com/2022/06/jenkins-controller-and-agents-architecture/>

Plugin Ecosystem

- Jenkins controller core is intentionally small
 - Most functionality comes from plugins
 - Thousands of plugins available
 - Examples
 - *Git / GitHub / GitLab integration*
 - *Pipeline features*
 - *Kubernetes agents*
 - *Credentials and RBAC*
 - Plugin Risks
 - *Plugins run with high privileges*
 - *Outdated plugins = security risk*
 - *Version compatibility matters*

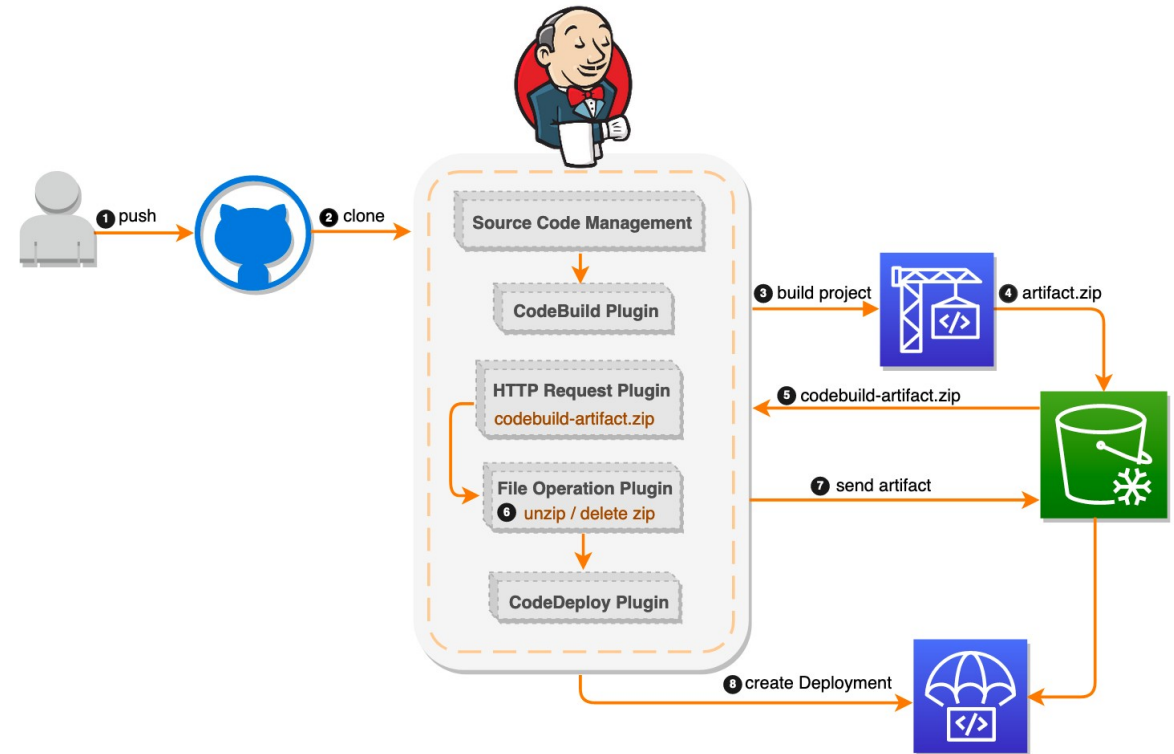


Image Credit: <https://thinkwithwp.com/blogs/devops/setting-up-a-ci-cd-pipeline-by-integrating-jenkins-with-aws-codebuild-and-aws-codedeploy/>

Security Model

- Authentication: local users, LDAP, SSO
- Authorization: role-based access control (RBAC)
 - Credentials stored centrally and injected securely
 - Fine-grained permissions per job, folder, or pipeline
- Governance considerations
 - Who can install plugins?
 - Who can modify pipelines?
 - How are credentials managed?
 - How are updates tested and rolled out?
- More detail in a later module

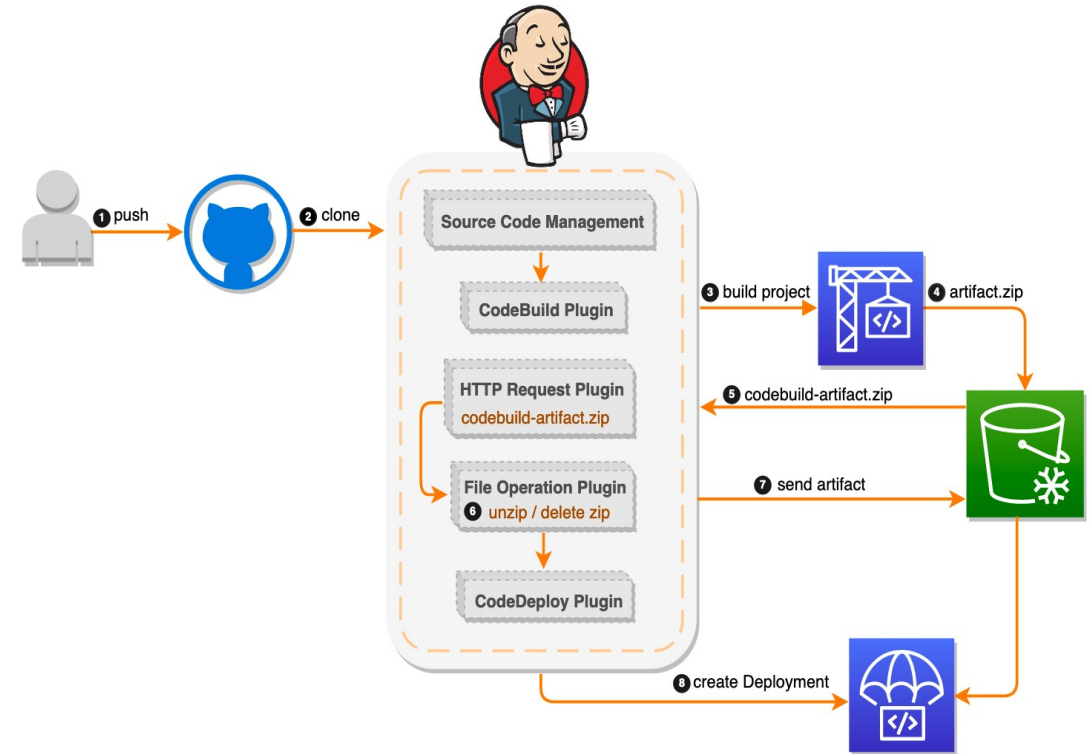


Image Credit: <https://thinkwithwp.com/blogs/devops/setting-up-a-ci-cd-pipeline-by-integrating-jenkins-with-aws-codebuild-and-aws-codedeploy/>

Controller/Agent Architecture

- Using agents provides
 - Separation of concerns
 - *For example, doing builds in different OS environments*
 - Scalability – can add more agents to manage workloads
 - Security isolation – each agent can have its own security profile
- Static agents
 - Always running
 - Simple but resource-heavy
 - Common in older Jenkins setups
 - Usually a physical host or a virtual machine

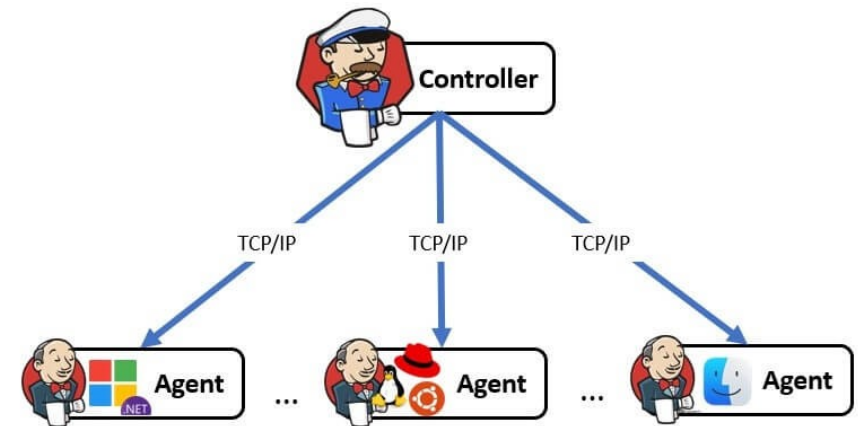


Image Credit: <https://naiwaen.debuggingsoft.com/2022/06/jenkins-controller-and-agents-architecture/>



Controller/Agent Architecture

- Ephemeral agents
 - Created on demand and destroyed after job completion
 - Often container-based or Kubernetes-based
 - Better isolation and scalability
 - Considered to be modern best practice
- Execution flow of ephemeral agents
 - SCM event triggers pipeline
 - Controller schedules work to be done
 - Agent spins up
 - Pipeline stages execute the work
 - Agent is destroyed

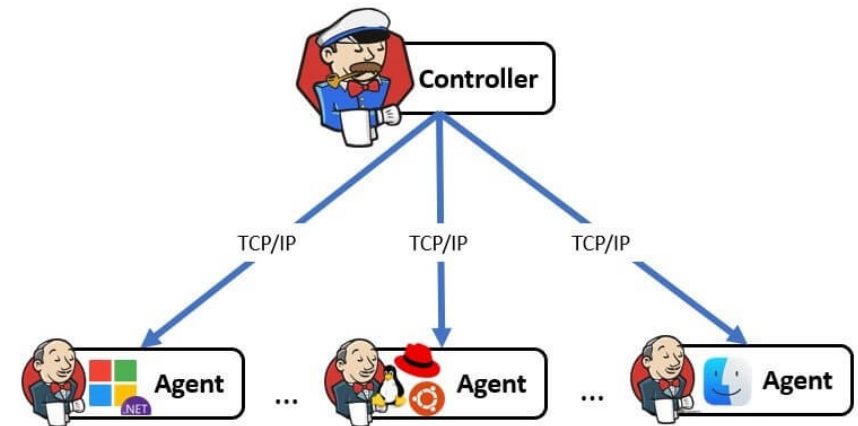


Image Credit: <https://naiwaen.debuggingsoft.com/2022/06/jenkins-controller-and-agents-architecture/>

SCM-Driven Automation

- Source control as the single source of truth
 - For both the application code and pipeline code (Jenkinsfile)
 - Pipelines ingest application code
 - Jenkinsfile is an IaC specification that defines the CI/CD steps and flow of execution
 - Jenkinsfile is versioned with the application
 - *Similar to how unit and integration tests are versioned*
 - Changes to Jenkinsfile code
 - *Follow code review processes*
 - *Subject to the same QA testing as application code*

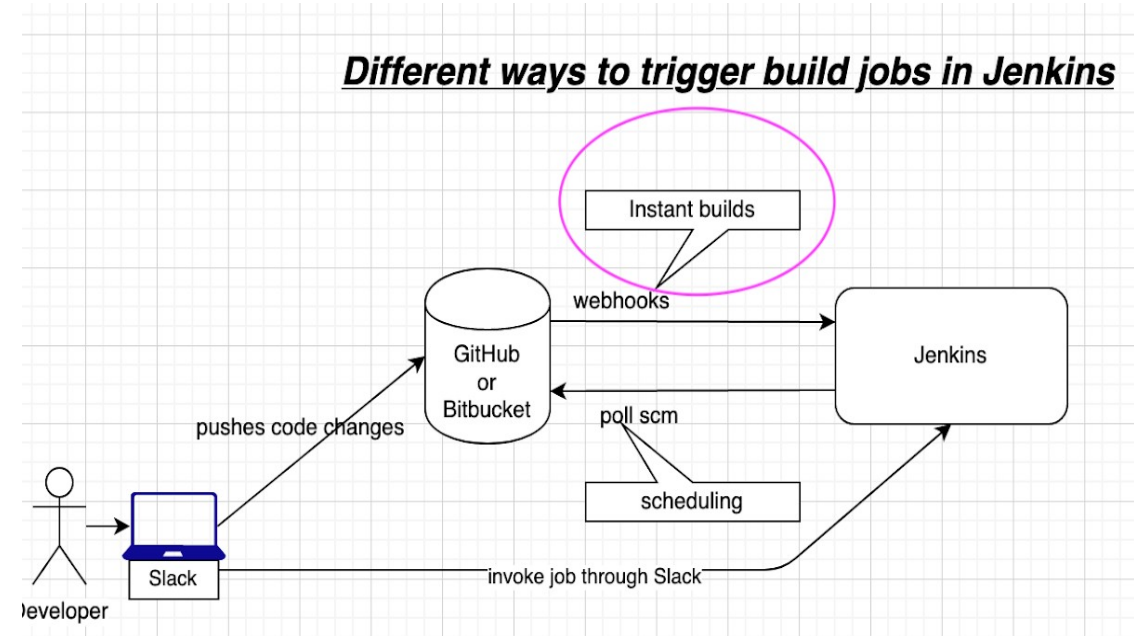


Image Credit: <https://www.coachdevops.com/2020/06/how-to-configure-webhooks-in-bitbucket.html>

SCM-Driven Automation

- Webhooks
 - GitHub/GitLab notify Jenkins on:
 - *Push*
 - *Pull/Merge Request*
 - *Tag creation*
 - Enables event-driven automation

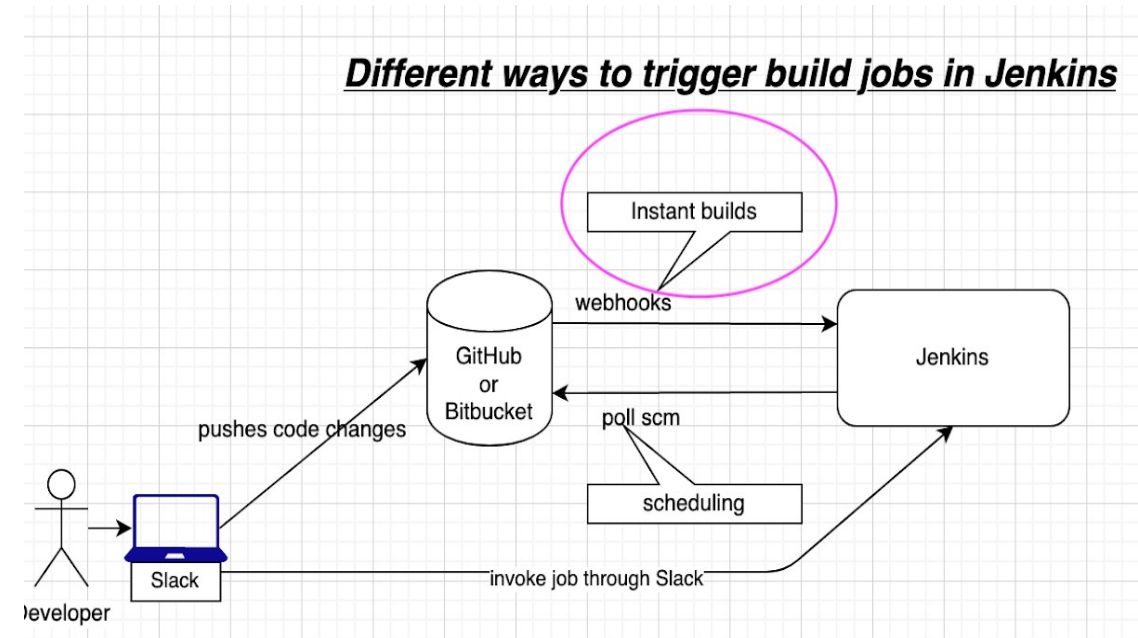
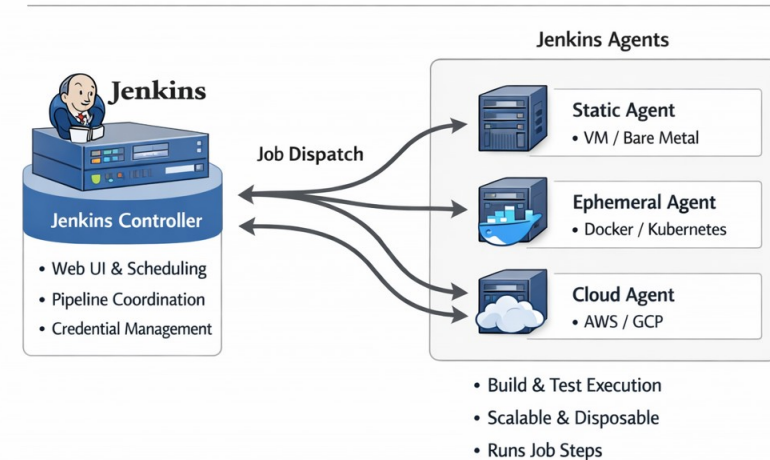


Image Credit: <https://www.coachdevops.com/2020/06/how-to-configure-webhooks-in-bitbucket.html>

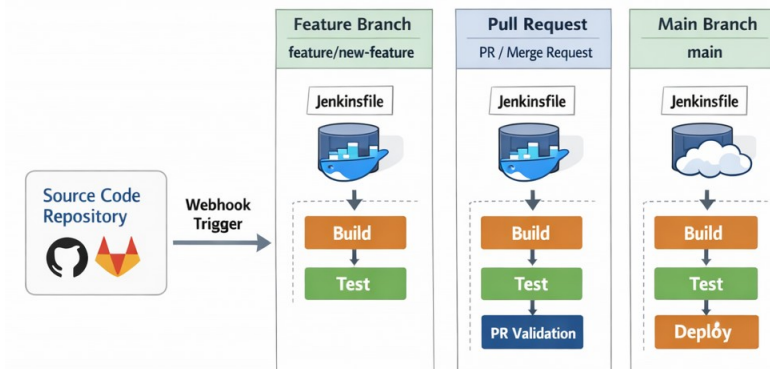
SCM-Driven Automation

- Multibranch Pipelines
 - Jenkins automatically discovers branches
 - Each branch can have its own Jenkinsfile
- Enables
 - Pull request validation
 - Feature branch testing
 - Main branch deployments

Jenkins Controller/Agent Architecture



Jenkins Multibranch Pipeline Flow



Jenkins vs GitHub Actions vs GitLab CI/CD

Feature	Jenkins	GitHub Actions	GitLab CI/CD
Hosting	Self-managed	SaaS	SaaS / Self
Pipeline Language	Jenkinsfile (Groovy)	YAML	YAML
Plugin Ecosystem	Very large	Marketplace actions	Built-in features
Customization	Extremely high	Moderate	High
Enterprise Control	Full	Limited	Strong



Jenkins vs GitHub Actions vs GitLab CI/CD

- Jenkins is preferred for
 - Complex, multi-tool workflows
 - Hybrid or on-prem environments
 - Advanced customization needs
 - Regulated industries
 - Long-running or specialized builds
- GitHub/GitLab are preferred for
 - Simple to moderate pipelines
 - Tight integration with SCM
 - Minimal infrastructure management
 - Fast onboarding



Installing Jenkins LTS

- Installing Jenkins means:
 - Installing the Jenkins controller
 - Running it as a long-lived service
 - Preparing it to manage jobs, pipelines, and agents
- Jenkins is a Java application distributed as
 - Native packages (RPM, DEB) or Windows MSI
 - WAR file
 - Docker image
- Always start with LTS unless you there is a reason for using the weekly build



Installing Jenkins LTS

- Native Package Install
 - Installed as a system service
 - Runs on a VM or server
 - Direct access to filesystem
 - Traditional enterprise model
- WAR File
 - Portable
 - Manual startup
 - Often used for testing or learning
- Docker
 - Containerized Jenkins controller



Installing Jenkins LTS

- What happens during first startup
 - Jenkins creates \$JENKINS_HOME
 - Default configuration files are generated
 - Initial admin password is created
 - Setup wizard is triggered

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

```
/var/jenkins_home/secrets/initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

Continue



Initial Setup Wizard

- On first startup, Jenkins
 - Locks itself
 - Requests an admin password
 - Prompts for plugin installation
 - Creates the first admin user
 - Sets basic configuration

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log ([not sure where to find it?](#)) and this file on the server:

`/var/jenkins_home/secrets/initialAdminPassword`

Please copy the password from either location and paste it below.

Administrator password

[Continue](#)



Plugin Selection

- Two common choices:
 - Suggested plugins
 - Custom plugin selection
- For most installations
 - Suggested plugins are sufficient
 - Plugins can always be added later
 - However, removing them is harder

The screenshot shows the 'Plugins' management page in a web application. On the left, there is a sidebar with four options: 'Updates', 'Available plugins' (which is highlighted), 'Installed plugins', and 'Advanced settings'. The main area is titled 'Plugins' and features a search bar with the text 'blue ocean'. Below the search bar is a table with four columns: 'Install', 'Name', and 'Released'. The table lists four plugins, each with an 'Install' checkbox, a name with a version number, and a 'Released' timestamp. The first plugin is 'Blue Ocean 1.27.3' with sub-plugins 'External Site/Tool Integrations' and 'User Interface', released '1 day 2 hr ago'. The second is 'Display URL for Blue Ocean 2.4.1' with the description 'This plugin generates BlueOcean specific URLs for the Display URL plugin.', released '2 yr 1 mo ago'. The third is 'Personalization for Blue Ocean 1.27.3' with sub-plugins 'External Site/Tool Integrations' and 'User Interface', released '1 day 2 hr ago'. The fourth is 'Bitbucket Pipeline for Blue Ocean 1.27.3' with the description 'BlueOcean Bitbucket pipeline creator', released '1 day 2 hr ago'. At the bottom of the table, there are two buttons: 'Install without restart' and 'Download now and install after restart'. To the right of these buttons, it says 'Update information obtained: 1 hr 18 min ago' and a 'Check now' button.

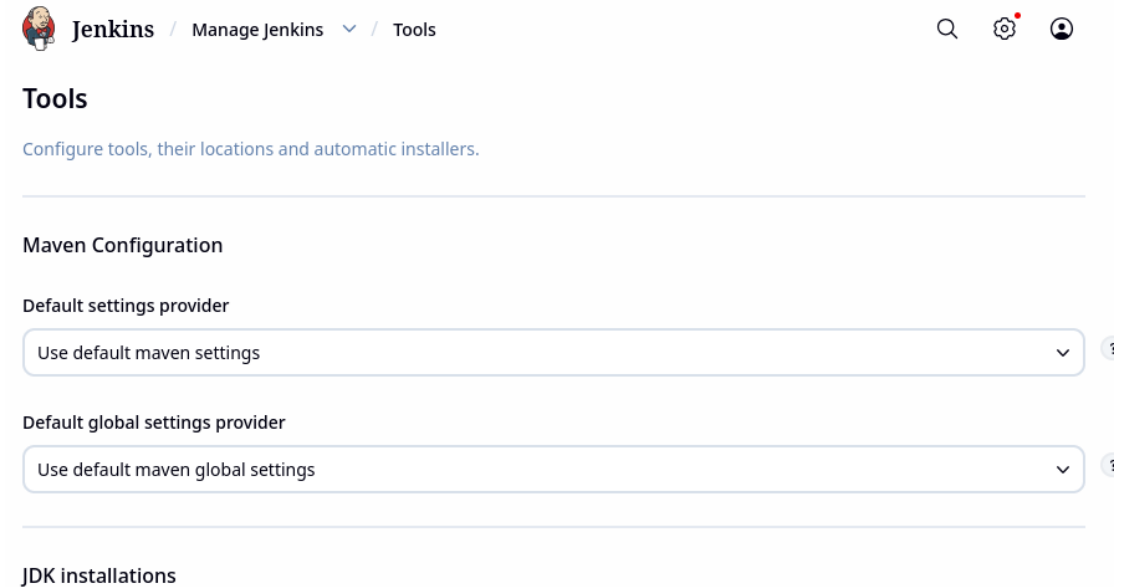
Install	Name ↓	Released
☐	Blue Ocean 1.27.3 External Site/Tool Integrations User Interface BlueOcean Aggregator	1 day 2 hr ago
☐	Display URL for Blue Ocean 2.4.1 This plugin generates BlueOcean specific URLs for the Display URL plugin.	2 yr 1 mo ago
☐	Personalization for Blue Ocean 1.27.3 External Site/Tool Integrations User Interface Blue Ocean Personalization	1 day 2 hr ago
☐	Bitbucket Pipeline for Blue Ocean 1.27.3 BlueOcean Bitbucket pipeline creator	1 day 2 hr ago

Install without restart Download now and install after restart Update information obtained: 1 hr 18 min ago Check now



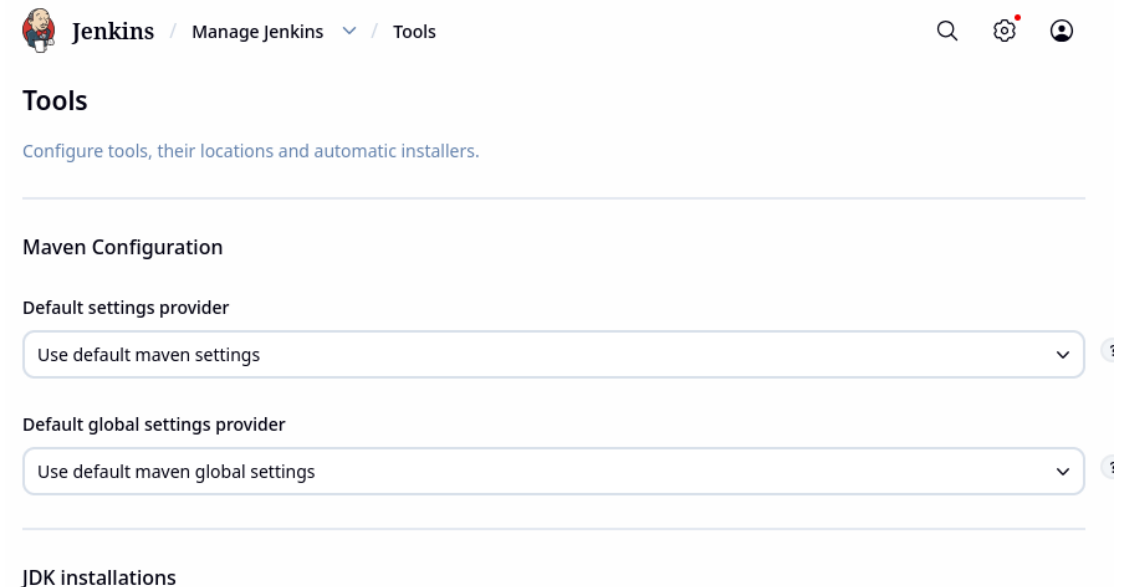
Global Tools Configuration

- Global tools:
 - Are *not* binaries
 - Are *not* installed on the controller
 - Are *not* automatically present on agents
 - Are named tool definitions
 - Are stored in Jenkins configuration
- Used by pipelines as references
 - Global tools are labels, not software



Reference Tools by Name

- In a Jenkinsfile:
 - “Use Maven version maven-3.9”
 - The name “maven-3.9” is defined globally in Jenkins
 - Abstracts away
 - *Version numbers*
 - *Installation paths*
 - *Agent OS differences*
- Benefits
 - Pipelines become portable
 - Jenkins controls versions centrally
 - Developers don’t hardcode paths



What Happens at Runtime

- Step 1: Pipeline Starts
 - Controller reads the Jenkinsfile
 - Sees: “This pipeline needs Maven maven-3.9”
- Step 2: Agent Is Selected
 - Controller assigns an agent
 - Agent may be:
 - A VM
 - A Docker container
 - A Kubernetes pod
- Step 3: Tool Resolution
 - Controller sends the tool definition to the agent
 - Definition includes:
 - Tool type (Maven, JDK, Node)
 - Version
 - Installation strategy

Maven installations

Maven installations ^

 Edited

+ Add Maven

 Maven

Name

Maven-3.9.11

MAVEN_HOME

/usr/share/maven

☐ Install automatically ?

+ Add Maven



What Happens at Runtime

- Step 4: Tool Installation (If Needed)
 - On the agent
 - If tool is already installed then use it
 - If not
 - *Jenkins downloads it*
 - *Installs it locally*
 - *Caches it for reuse (if agent persists)*
- Step 5: Tool Execution
 - Agent updates PATH and environment variables
 - Pipeline steps run
 - Controller receives logs and results

Maven installations

Maven installations ^

 Edited

+ Add Maven

⋮ Maven

Name

Maven-3.9.11

MAVEN_HOME

/usr/share/maven

☐ Install automatically ?

+ Add Maven



What Auto-Install Means

- Jenkins can download tools automatically
- Controlled by:
 - Tool definition settings
 - Plugin support
 - Agent permissions
- Benefits
 - No manual agent setup
 - New agents work immediately
 - Ephemeral agents become practical
- Pitfalls
 - Builds depend on external downloads
 - Tool versions can change unexpectedly
 - Network failures can break builds

Maven installations

Maven installations ^

 Edited

+ Add Maven

⋮ Maven

Name

Maven-3.9.11

MAVEN_HOME

/usr/share/maven

☐ Install automatically ?

+ Add Maven



Ensuring Consistent Builds

- Without global tools
 - Each agent might have different versions
 - Pipelines hardcode paths
 - “Works on my agent” problems appear
- With global tools
 - Jenkins enforces version consistency
 - Pipelines are environment-agnostic
 - Upgrades are centralized and controlled
- Critical when
 - Multiple teams share Jenkins
 - Agents are ephemeral
 - Jenkins uses containers or Kubernetes

```
pipeline {
  agent any

  tools {
    // These refer to Global Tool Configuration names
    jdk 'JDK17'
    maven 'Maven3'
  }

  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/example/my-java-app.git'
      }
    }

    stage('Build') {
      steps {
        sh 'mvn clean package'
      }
    }
  }
}
```



Controller vs Agent Tooling

- Controller responsibilities
 - Store tool definitions
 - Define names and versions
 - Decide what is needed
 - Never run builds
- Agent responsibilities
 - Install tools
 - Execute commands
 - Run builds and tests
 - Clean up afterward
- The controller is the planner
- The agent is the worker

```
pipeline {
  agent any

  tools {
    // These refer to Global Tool Configuration names
    jdk 'JDK17'
    maven 'Maven3'
  }

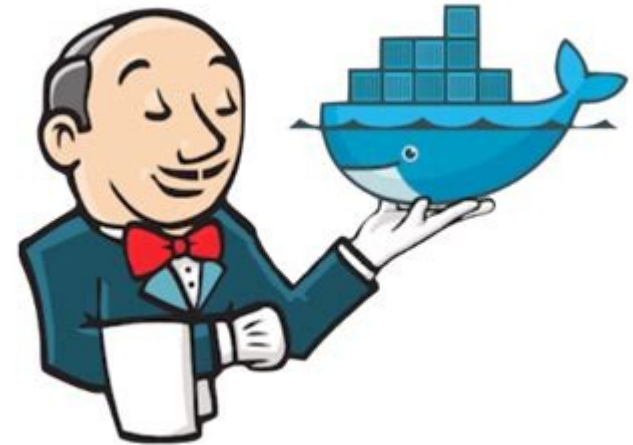
  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/example/my-java-app.git'
      }
    }

    stage('Build') {
      steps {
        sh 'mvn clean package'
      }
    }
  }
}
```



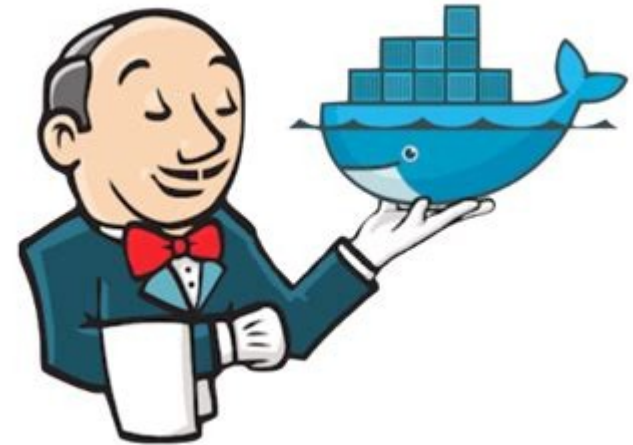
Running Jenkins in Docker

- Docker helps solve common Jenkins problems
 - Consistent environments
 - Easier upgrades
 - Simplified rollback
 - Faster setup for labs and training
- Jenkins in Docker is NOT Jenkins as a stateless service
 - The controller still needs persistent storage



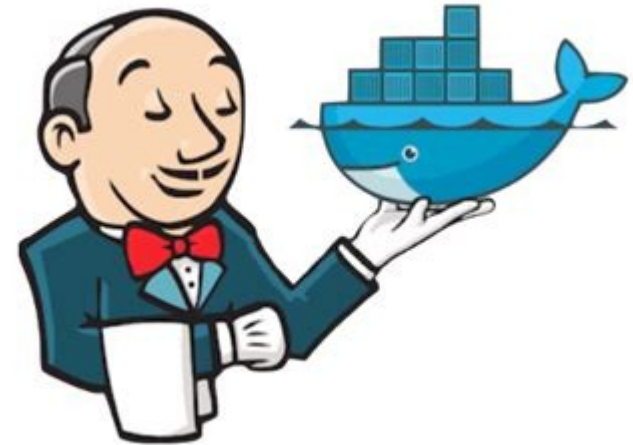
Running Jenkins in Docker

- Persistent volumes
 - Jenkins state must survive container restarts
 - `$JENKINS_HOME` is mounted to a volume
 - Losing this volume = losing Jenkins
- Container Lifecycle
 - Container can be destroyed and recreated
 - Jenkins data remains intact
 - Encourages clean operational practices



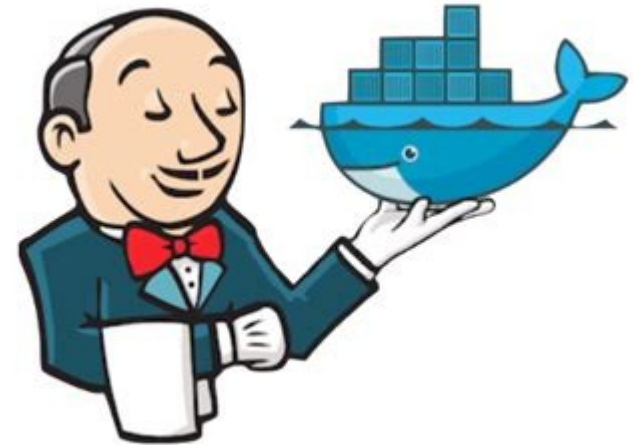
Running Jenkins in Docker

- Running Jenkins in Docker means
 - Jenkins runs inside a container
 - Jenkins data lives in a mounted volume
 - The container can be replaced without losing Jenkins state
 - Provides control, consistency, and safety without changing Jenkins' architecture



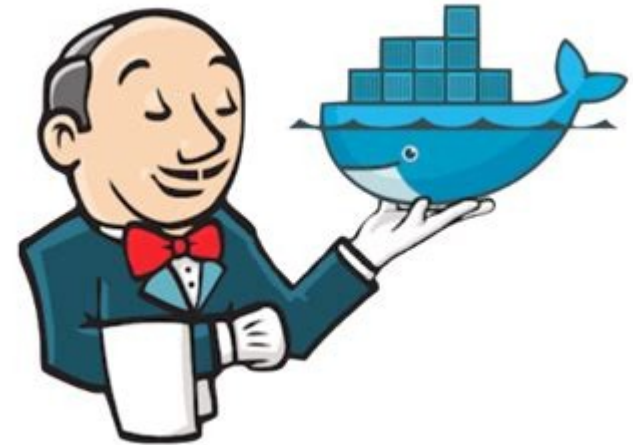
Docker: Fast and Predictable Setup

- Without Docker
 - Install Java
 - Configure OS packages
 - Manage service startup
 - Troubleshoot environment differences
- With Docker
 - Pull the Jenkins LTS image
 - Start a container
 - Jenkins is immediately usable
 - Everyone gets the same Jenkins
 - Avoids setup chaos



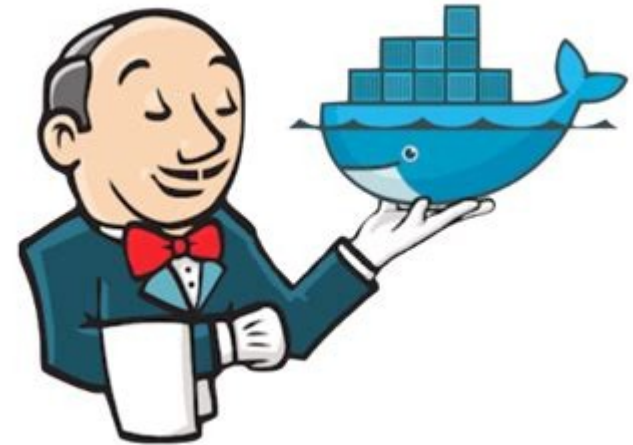
Docker: Environment Consistency

- Docker ensures
 - Same Jenkins version
 - Same Java version
 - Same default configuration
 - Same filesystem layout
- Eliminates
 - “Works on my machine”
 - OS-specific surprises
 - Hidden dependencies



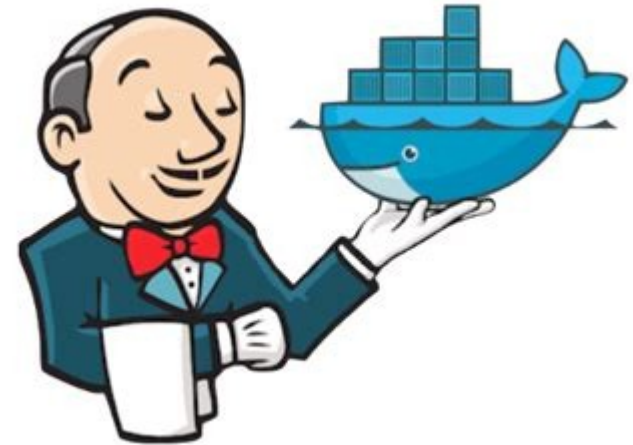
Docker: Modern Jenkins Practices

- Docker aligns naturally with
 - Configuration-as-Code (JCasC)
 - Ephemeral agents
 - Cloud deployments
 - Kubernetes migration



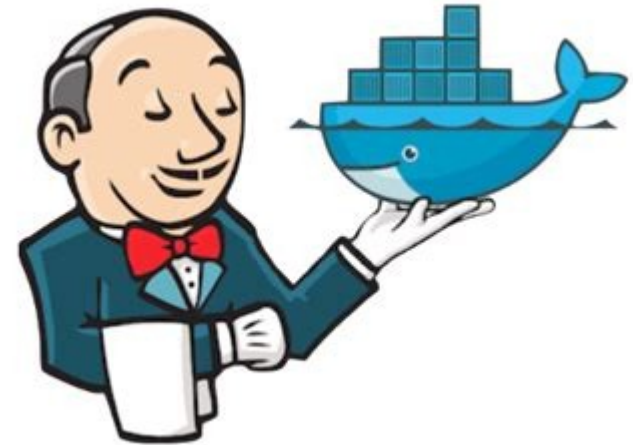
Docker: Reduced Operational Risk

- Avoids common Jenkins failures
 - Broken Java upgrades
 - OS package conflicts
 - Accidental system changes
 - Hard-to-reproduce environments
- Important because
 - Jenkins controllers are long-running
 - Downtime is disruptive
 - Recovery needs to be predictable



When to Avoid Docker

- Very high-scale enterprise setups
- Strict compliance environments
- Existing Jenkins infrastructure already standardized



Configuration-as-Code (JCasC)

- Configuration-as-Code (JCasC) is where
 - Jenkins configurations are defined in YAML files
 - Loads the configuration automatically at startup
 - Treats Jenkins configuration like application code
- Declarative
 - Describes how Jenkins should be configured
 - Replaces configuring Jenkins through the UI

```
jenkins:
  systemMessage: "Welcome to the Jenkins Training Instance"

  securityRealm:
    local:
      allowsSignup: false
      users:
        - id: admin
          password: admin123

  authorizationStrategy:
    loggedInUsersCanDoAnything:
      allowAnonymousRead: false

  tool:
    git:
      installations:
        - name: Default Git
          home: /usr/bin/git
```



Configuration-as-Code (JCasC)

- JcasC, can define
 - Security settings
 - Authentication and authorization
 - Users and roles
 - Credentials
 - Global tools (JDK, Maven, Node, etc.)
 - Plugin configuration
 - System settings
- But not
 - Individual pipeline logic
 - Build steps inside Jenkinsfiles
 - Pipelines are code, JCasC is configuration.

```
jenkins:
  systemMessage: "Welcome to the Jenkins Training Instance"

  securityRealm:
    local:
      allowsSignup: false
      users:
        - id: admin
          password: admin123

  authorizationStrategy:
    loggedInUsersCanDoAnything:
      allowAnonymousRead: false

  tool:
    git:
      installations:
        - name: Default Git
          home: /usr/bin/git
```



Configuration-as-Code (JCasC)

- JCasC YAML files are stored on disk
- Often mounted into Jenkins via:
 - Docker volumes
 - ConfigMaps (Kubernetes)
- When JCasC Is applied
 - Jenkins startup
 - Configuration reload (optional)
- What Jenkins does
 - Jenkins starts
 - JCasC plugin reads YAML
 - Jenkins configures itself accordingly
 - UI reflects the defined state

```
jenkins:
  systemMessage: "Welcome to the Jenkins Training Instance"

  securityRealm:
    local:
      allowsSignup: false
      users:
        - id: admin
          password: admin123

  authorizationStrategy:
    loggedInUsersCanDoAnything:
      allowAnonymousRead: false

tool:
  git:
    installations:
      - name: Default Git
        home: /usr/bin/git
```



Benefits

- Makes Jenkins reproducible
- Supports automation and version control
- Enables safer upgrades
- Snowflake Jenkins instances that
 - Were configured manually over time
 - Have unique, undocumented settings
 - Cannot be easily recreated
 - No one fully understands anymore
 - Breaks when you try to upgrade or migrate
- Called a snowflake because it's unique and melts under pressure

```
jenkins:
  systemMessage: "Welcome to the Jenkins Training Instance"

  securityRealm:
    local:
      allowsSignup: false
      users:
        - id: admin
          password: admin123

  authorizationStrategy:
    loggedInUsersCanDoAnything:
      allowAnonymousRead: false

  tool:
    git:
      installations:
        - name: Default Git
          home: /usr/bin/git
```



Snowflake Instances

- Causes
 - Clicking through the UI to configure Jenkins
 - Installing plugins “just to try something”
 - Making emergency fixes directly on the server
 - Manually editing config files
 - Installing tools directly on agents
 - No version control for configuration
- Over time
 - The original admins leave
 - Configuration knowledge is lost

```
jenkins:
  systemMessage: "Welcome to the Jenkins Training Instance"

  securityRealm:
    local:
      allowsSignup: false
      users:
        - id: admin
          password: admin123

  authorizationStrategy:
    loggedInUsersCanDoAnything:
      allowAnonymousRead: false

tool:
  git:
    installations:
      - name: Default Git
        home: /usr/bin/git
```



Questions

